



**"RANKED 45
IN INDIA**

#University Category



**WORLD
UNIVERSITY
RANKINGS**

NO.1 PVT. UNIVERSITY IN
ACADEMIC REPUTATION IN INDIA



ACCREDITED GRADE 'A'
BY NAAC



PERFECT SCORE OF **150/150** AS A TESTAMENT
TO EXCEPTIONAL E-LEARNING METHODS

1



Applied Machine Learning

Unit 01 – Lecture 01: Introduction to ML & the Python ML Stack

Tofik Ali

School of Computer Science, UPES Dehradun

Autumn 2026

Topic focus: **What is machine learning?** what kinds there are, and the tools we will use

Repository: <https://github.com/tali7c/Applied-Machine-Learning>

Sources: Müller and Guido, *Introduction to Machine Learning with Python*, Ch. 1.; Bishop, *Pattern Recognition and Machine Learning*, Ch. 1.; Murphy, *Probabilistic Machine Learning: An Introduction*, MIT Press, 2022, Ch. 1.; Mitchell, *Machine Learning*, McGraw Hill, 1997, Ch. 1.; Scikit-learn documentation, accessed 2026-08-04.



Quick Links

[Course](#)

[What is ML](#)

[Three Families](#)

[Generalisation](#)

[Python Stack](#)

[Summary](#)

Agenda

Course Overview

What is Machine Learning?

The Three Families

The Central Difficulty: Generalisation

The Python ML Stack

Summary

How this lecture works

Every idea appears twice

1. **By hand** — small numbers, on paper, so you see the mechanism.
2. **In code** — the same thing, so you see the library does exactly that and nothing magic.

Predict before you look

Several slides ask you to guess a number before it is revealed. **Commit to an answer.** Being wrong and knowing why is how this material sticks; watching it scroll past is not.

Full detail, worked solutions and further reading are in the notes: `notes.pdf`

Learning Outcomes

By the end of this lecture you should be able to:

- Explain what **machine learning** is, and spot problems where it is the wrong tool.

Mapped to **CO1** (core concepts) and **CO2** (build models with standard libraries).

Learning Outcomes

By the end of this lecture you should be able to:

- Explain what **machine learning** is, and spot problems where it is the wrong tool.
- State Mitchell's definition and apply it: name the **task**, **performance measure** and **experience**.

Mapped to **CO1** (core concepts) and **CO2** (build models with standard libraries).

Learning Outcomes

By the end of this lecture you should be able to:

- Explain what **machine learning** is, and spot problems where it is the wrong tool.
- State Mitchell's definition and apply it: name the **task**, **performance measure** and **experience**.
- Classify an unfamiliar problem as **supervised**, **unsupervised** or **reinforcement**.

Mapped to **CO1** (core concepts) and **CO2** (build models with standard libraries).

Learning Outcomes

By the end of this lecture you should be able to:

- Explain what **machine learning** is, and spot problems where it is the wrong tool.
- State Mitchell's definition and apply it: name the **task**, **performance measure** and **experience**.
- Classify an unfamiliar problem as **supervised**, **unsupervised** or **reinforcement**.
- Explain why fitting the training data is easy and **generalisation** is the real goal.

Mapped to **CO1** (core concepts) and **CO2** (build models with standard libraries).

Learning Outcomes

By the end of this lecture you should be able to:

- Explain what **machine learning** is, and spot problems where it is the wrong tool.
- State Mitchell's definition and apply it: name the **task**, **performance measure** and **experience**.
- Classify an unfamiliar problem as **supervised**, **unsupervised** or **reinforcement**.
- Explain why fitting the training data is easy and **generalisation** is the real goal.
- Fit and evaluate a model with **NumPy**, **pandas** and **scikit-learn**.

Mapped to **CO1** (core concepts) and **CO2** (build models with standard libraries).

Course Roadmap (Units I–VII)

- **Unit I:** Introduction — what ML is, its types, the Python stack.

Course Roadmap (Units I–VII)

- **Unit I:** Introduction — what ML is, its types, the Python stack.
- **Unit II:** Loss functions — MSE, MAE, Huber, cross-entropy, hinge, triplet.

Course Roadmap (Units I–VII)

- **Unit I:** Introduction — what ML is, its types, the Python stack.
- **Unit II:** Loss functions — MSE, MAE, Huber, cross-entropy, hinge, triplet.
- **Unit III:** Optimizers — SGD, mini-batch, momentum, Adagrad, RMSprop, Adam.

Course Roadmap (Units I–VII)

- **Unit I:** Introduction — what ML is, its types, the Python stack.
- **Unit II:** Loss functions — MSE, MAE, Huber, cross-entropy, hinge, triplet.
- **Unit III:** Optimizers — SGD, mini-batch, momentum, Adagrad, RMSprop, Adam.
- **Unit IV:** Data preprocessing — cleaning, features, splitting, cross-validation.

Course Roadmap (Units I–VII)

- **Unit I:** Introduction — what ML is, its types, the Python stack.
- **Unit II:** Loss functions — MSE, MAE, Huber, cross-entropy, hinge, triplet.
- **Unit III:** Optimizers — SGD, mini-batch, momentum, Adagrad, RMSprop, Adam.
- **Unit IV:** Data preprocessing — cleaning, features, splitting, cross-validation.
- **Unit V:** Regression — linear, logistic, multiple, regularization.

Course Roadmap (Units I–VII)

- **Unit I:** Introduction — what ML is, its types, the Python stack.
- **Unit II:** Loss functions — MSE, MAE, Huber, cross-entropy, hinge, triplet.
- **Unit III:** Optimizers — SGD, mini-batch, momentum, Adagrad, RMSprop, Adam.
- **Unit IV:** Data preprocessing — cleaning, features, splitting, cross-validation.
- **Unit V:** Regression — linear, logistic, multiple, regularization.
- **Unit VI:** Classification — KNN, trees, Bayes, ensembles, neural nets, SVM.

Course Roadmap (Units I–VII)

- **Unit I:** Introduction — what ML is, its types, the Python stack.
- **Unit II:** Loss functions — MSE, MAE, Huber, cross-entropy, hinge, triplet.
- **Unit III:** Optimizers — SGD, mini-batch, momentum, Adagrad, RMSprop, Adam.
- **Unit IV:** Data preprocessing — cleaning, features, splitting, cross-validation.
- **Unit V:** Regression — linear, logistic, multiple, regularization.
- **Unit VI:** Classification — KNN, trees, Bayes, ensembles, neural nets, SVM.
- **Unit VII:** Clustering — hierarchical, K-means, K-medoids, CLARA, DBSCAN.

Course Roadmap (Units I–VII)

- **Unit I:** Introduction — what ML is, its types, the Python stack.
- **Unit II:** Loss functions — MSE, MAE, Huber, cross-entropy, hinge, triplet.
- **Unit III:** Optimizers — SGD, mini-batch, momentum, Adagrad, RMSprop, Adam.
- **Unit IV:** Data preprocessing — cleaning, features, splitting, cross-validation.
- **Unit V:** Regression — linear, logistic, multiple, regularization.
- **Unit VI:** Classification — KNN, trees, Bayes, ensembles, neural nets, SVM.
- **Unit VII:** Clustering — hierarchical, K-means, K-medoids, CLARA, DBSCAN.

Why this order

Units II and III come early on purpose. Once you know **what quantity we minimise** and **how**, every method in Units V–VII is a variation on one theme rather than a new thing to memorise.

Course Structure and Assessment

Course code	CSAI2017P
L-T-P-C	4-0-1-5
Units	7
Theory	24 lecture packages
Lab	15 experiments / 10 assignments

Component	Marks
Theory assignments (2, each with a test)	5
Theory quizzes (10 × 40 min)	10
Project — individual quiz	10
Project — group submission state	5
Lab assignments (10)	10
Lab quizzes (10 × 60 min)	10
Mid-Semester examination	20
End-Semester examination	30
Total	100

Read this carefully

The project is worth 15, and **10 of those 15 are individual**. A strong contributor in a weak group can still score well; a passenger cannot.

Indicative. The final split is subject to departmental approval and will be confirmed on the LMS.

Test-0: a diagnostic, not an exam

- A **30-minute** diagnostic on prerequisites and general understanding. **Date to be announced.**

Test-0: a diagnostic, not an exam

- A **30-minute** diagnostic on prerequisites and general understanding. **Date to be announced.**
- It **carries no marks** and is not part of any component.

Test-0: a diagnostic, not an exam

- A **30-minute** diagnostic on prerequisites and general understanding. **Date to be announced.**
- It **carries no marks** and is not part of any component.
- Its purpose: to let the teaching team see **how you approach a problem**, so the short tests through the semester suit how you work.

Test-0: a diagnostic, not an exam

- A **30-minute** diagnostic on prerequisites and general understanding. **Date to be announced.**
- It **carries no marks** and is not part of any component.
- Its purpose: to let the teaching team see **how you approach a problem**, so the short tests through the semester suit how you work.
- It is **not** a ranking and **not** an ability test. Nothing to revise.

Test-0: a diagnostic, not an exam

- A **30-minute** diagnostic on prerequisites and general understanding. **Date to be announced.**
- It **carries no marks** and is not part of any component.
- Its purpose: to let the teaching team see **how you approach a problem**, so the short tests through the semester suit how you work.
- It is **not** a ranking and **not** an ability test. Nothing to revise.

How to answer it

Show your working. An incomplete answer that makes its reasoning visible is **more** useful to us than a bare correct one. If you do not know something, write what you would need to find out.

The Shape of the Problem

Programs we *can* write by hand

Sort a list. Parse a date. Compute a payroll.

We know the rule. We encode the rule.
Correct by construction.

Programs we *cannot*

Is this email spam? Which digit is this?
Does this scan show a tumour?

We **recognise** the answer but cannot **state the rule**.

The Shape of the Problem

Programs we *can* write by hand

Sort a list. Parse a date. Compute a payroll.

We know the rule. We encode the rule.
Correct by construction.

Programs we *cannot*

Is this email spam? Which digit is this?
Does this scan show a tumour?

We **recognise** the answer but cannot **state the rule**.

Try it now: write a precise rule that tells a handwritten **3** from an **8**, covering every style in this room.

The Shape of the Problem

Programs we *can* write by hand

Sort a list. Parse a date. Compute a payroll.

We know the rule. We encode the rule.
Correct by construction.

Programs we *cannot*

Is this email spam? Which digit is this?
Does this scan show a tumour?

We **recognise** the answer but cannot **state the rule**.

Try it now: write a precise rule that tells a handwritten **3** from an **8**, covering every style in this room.

Working idea for this course

ML is what you reach for when you have **examples of the answer** but not a **statement of the rule**.

That is the *supervised* case — most of this course. Two other families relax it; we get there shortly.

Two Ways to Build a Spam Filter

Rule-based

```
def classify(subject, sender, bangs):  
    if "free money" in subject:  
        return "SPAM"  
    if sender in blocklist:  
        return "SPAM"  
    if bangs > 5:  
        return "SPAM"  
    return "HAM"
```

Monday: works.

Tuesday: they write fr33 m0ney. You edit code.

Wednesday: they pad with normal words. You edit code.

Learned

```
model.fit(emails, labels)  
model.predict(new_email)
```

They adapt. Users press "spam". You **refit**.

Two Ways to Build a Spam Filter

Rule-based

```
def classify(subject, sender, bangs):  
    if "free money" in subject:  
        return "SPAM"  
    if sender in blocklist:  
        return "SPAM"  
    if bangs > 5:  
        return "SPAM"  
    return "HAM"
```

Monday: works.

Tuesday: they write fr33 m0ney. You edit code.

Wednesday: they pad with normal words. You edit code.

Learned

```
model.fit(emails, labels)  
model.predict(new_email)
```

They adapt. Users press "spam". You **refit**.

The bottleneck moved off *you*: adaptation became a **data** problem, not a **programming** problem.

Two Ways to Build a Spam Filter

Rule-based

```
def classify(subject, sender, bangs):  
    if "free money" in subject:  
        return "SPAM"  
    if sender in blocklist:  
        return "SPAM"  
    if bangs > 5:  
        return "SPAM"  
    return "HAM"
```

Monday: works.

Tuesday: they write fr33 m0ney. You edit code.

Wednesday: they pad with normal words. You edit code.

But you gave something up — what?

The rule-based filter can *tell you why it fired*. That is **interpretability**. Unit VI recovers some of it with decision trees.

Learned

```
model.fit(emails, labels)  
model.predict(new_email)
```

They adapt. Users press "spam". You **refit**.

The bottleneck moved off *you*: adaptation became a **data** problem, not a **programming** problem.

A Working Definition (Mitchell, 1997)

Definition

A computer program is said to **learn** from experience E with respect to a class of tasks T and performance measure P , if its performance at tasks in T , as measured by P , improves with experience E .

	Task T	Experience E	Performance P
Spam filter	classify email	labelled inbox	
House prices	predict price	past sales records	
Chess engine	choose a move	games played	

A Working Definition (Mitchell, 1997)

Definition

A computer program is said to **learn** from experience E with respect to a class of tasks T and performance measure P , if its performance at tasks in T , as measured by P , improves with experience E .

	Task T	Experience E	Performance P
Spam filter	classify email	labelled inbox	fraction correct
House prices	predict price	past sales records	
Chess engine	choose a move	games played	

A Working Definition (Mitchell, 1997)

Definition

A computer program is said to **learn** from experience E with respect to a class of tasks T and performance measure P , if its performance at tasks in T , as measured by P , improves with experience E .

	Task T	Experience E	Performance P
Spam filter	classify email	labelled inbox	fraction correct
House prices	predict price	past sales records	mean absolute error, in rupees
Chess engine	choose a move	games played	

A Working Definition (Mitchell, 1997)

Definition

A computer program is said to **learn** from experience E with respect to a class of tasks T and performance measure P , if its performance at tasks in T , as measured by P , improves with experience E .

	Task T	Experience E	Performance P
Spam filter	classify email	labelled inbox	fraction correct
House prices	predict price	past sales records	mean absolute error, in rupees
Chess engine	choose a move	games played	win rate

A Working Definition (Mitchell, 1997)

Definition

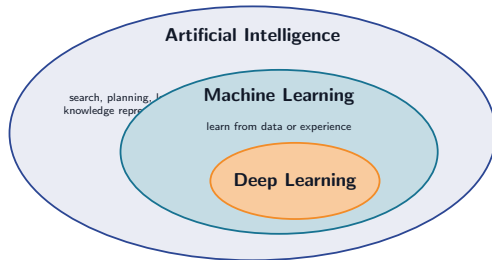
A computer program is said to **learn** from experience E with respect to a class of tasks T and performance measure P , if its performance at tasks in T , as measured by P , improves with experience E .

	Task T	Experience E	Performance P
Spam filter	classify email	labelled inbox	fraction correct
House prices	predict price	past sales records	mean absolute error, in rupees
Chess engine	choose a move	games played	win rate

P is the hard column

“Fraction correct” sounds obvious — until you notice that a filter which passes *everything* scores 97% when only 3% of mail is spam. Most failed ML projects failed here, not at the modelling.

Where ML Sits: $AI \supset ML \supset DL$



Deep learning is a subset, not an upgrade

On tabular data of the size you meet in this course, a gradient-boosted tree usually **matches or beats** a neural network, trains in seconds, and needs far less tuning. Choosing the smallest model that solves the problem is a skill, not a compromise.

Why Now? Most of the Mathematics is Old

- Least squares — Legendre, **1805**. Perceptron — **1958**. Backprop in use — **1980s**.

Why Now? Most of the Mathematics is Old

- Least squares — Legendre, **1805**. Perceptron — **1958**. Backprop in use — **1980s**.
- **Data**. Transactions, sensors, images and logs are digital and cheap to store. Examples now exist as a by-product of ordinary business.

Why Now? Most of the Mathematics is Old

- Least squares — Legendre, **1805**. Perceptron — **1958**. Backprop in use — **1980s**.
- **Data**. Transactions, sensors, images and logs are digital and cheap to store. Examples now exist as a by-product of ordinary business.
- **Compute**. Parallel hardware made the arithmetic of Unit III affordable at scale.

Why Now? Most of the Mathematics is Old

- Least squares — Legendre, **1805**. Perceptron — **1958**. Backprop in use — **1980s**.
- **Data**. Transactions, sensors, images and logs are digital and cheap to store. Examples now exist as a by-product of ordinary business.
- **Compute**. Parallel hardware made the arithmetic of Unit III affordable at scale.
- **Algorithms and tooling**. Better optimisers and regularisation — and libraries that put all of it three lines away.

Why Now? Most of the Mathematics is Old

- Least squares — Legendre, **1805**. Perceptron — **1958**. Backprop in use — **1980s**.
- **Data**. Transactions, sensors, images and logs are digital and cheap to store. Examples now exist as a by-product of ordinary business.
- **Compute**. Parallel hardware made the arithmetic of Unit III affordable at scale.
- **Algorithms and tooling**. Better optimisers and regularisation — and libraries that put all of it three lines away.

What this means for you

Much of what you will do this semester would have been a PhD project in 1995 and is now an `import` statement. So the course spends its time on **judgement** — which method, which metric, is this result honest — not on implementation.

When Not to Use Machine Learning

- **The rule is known and stable.** Income tax is a published formula. A model that gets it 99% right is *worse* than arithmetic.

When Not to Use Machine Learning

- **The rule is known and stable.** Income tax is a published formula. A model that gets it 99% right is *worse* than arithmetic.
- **No labelled examples**, and no realistic way to get them.

When Not to Use Machine Learning

- **The rule is known and stable.** Income tax is a published formula. A model that gets it 99% right is *worse* than arithmetic.
- **No labelled examples**, and no realistic way to get them.
- **Errors are unacceptable** and you cannot bound them. A model gives a rate, not a guarantee.

When Not to Use Machine Learning

- **The rule is known and stable.** Income tax is a published formula. A model that gets it 99% right is *worse* than arithmetic.
- **No labelled examples**, and no realistic way to get them.
- **Errors are unacceptable** and you cannot bound them. A model gives a rate, not a guarantee.
- **Every decision must be explained** to a regulator, and a simple rule would nearly do.

When Not to Use Machine Learning

- **The rule is known and stable.** Income tax is a published formula. A model that gets it 99% right is *worse* than arithmetic.
- **No labelled examples**, and no realistic way to get them.
- **Errors are unacceptable** and you cannot bound them. A model gives a rate, not a guarantee.
- **Every decision must be explained** to a regulator, and a simple rule would nearly do.
- **Training data will not resemble deployment.** A model trained on last year's customers predicts last year's customers.

When Not to Use Machine Learning

- **The rule is known and stable.** Income tax is a published formula. A model that gets it 99% right is *worse* than arithmetic.
- **No labelled examples**, and no realistic way to get them.
- **Errors are unacceptable** and you cannot bound them. A model gives a rate, not a guarantee.
- **Every decision must be explained** to a regulator, and a simple rule would nearly do.
- **Training data will not resemble deployment.** A model trained on last year's customers predicts last year's customers.

Discussion

A hospital wants to predict which patients will miss an appointment, to overbook.

Which of the five bites hardest?

One Question Decides Everything

Do you already have the answers?

Yes, for past examples

Supervised

learn a mapping

$x \mapsto y$

labelled pairs $\{(x_i, y_i)\}$

No, but I want structure

Unsupervised

find structure

in x alone

unlabelled $\{x_i\}$

*No, but I can act
and be scored*

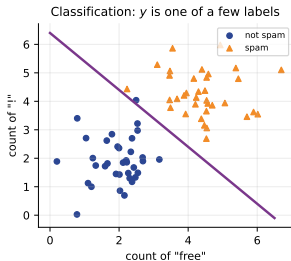
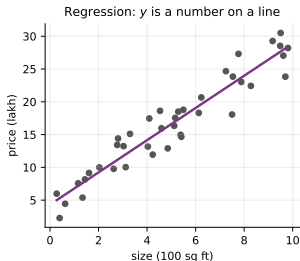
Reinforcement

learn a policy

from reward

environment + reward

Supervised: Regression vs Classification



Regression — y is a number.
Price, temperature, time to failure.
Unit V; lab experiments 3–4.

Classification — y is a label.
Spam/ham, digit 0–9, benign/malignant.
Unit VI; lab experiments 5–9, 11–12.

The distinction is about the **target**, not the features or the method.

By Hand: 1-Nearest Neighbour

Five training flowers, measured as (petal length, petal width) in cm:

length	width	species
1.4	0.2	setosa
1.3	0.2	setosa
4.7	1.4	versicolor
4.5	1.5	versicolor
6.0	2.5	virginica

New flower: (4.6, 1.4)

By Hand: 1-Nearest Neighbour

Five training flowers, measured as (petal length, petal width) in cm:

length	width	species
1.4	0.2	setosa
1.3	0.2	setosa
4.7	1.4	versicolor
4.5	1.5	versicolor
6.0	2.5	virginica

New flower: (4.6, 1.4)

$$d = \sqrt{(a_1 - b_1)^2 + (a_2 - b_2)^2}$$

Working	Distance
$\sqrt{3.2^2 + 1.2^2}$	3.42
$\sqrt{3.3^2 + 1.2^2}$	3.51
$\sqrt{0.1^2 + 0.0^2}$	0.10
$\sqrt{0.1^2 + 0.1^2}$	0.14
$\sqrt{1.4^2 + 1.1^2}$	1.78

By Hand: 1-Nearest Neighbour

Five training flowers, measured as (petal length, petal width) in cm:

length	width	species
1.4	0.2	setosa
1.3	0.2	setosa
4.7	1.4	versicolor
4.5	1.5	versicolor
6.0	2.5	virginica

New flower: (4.6, 1.4)

$$d = \sqrt{(a_1 - b_1)^2 + (a_2 - b_2)^2}$$

Working	Distance
$\sqrt{3.2^2 + 1.2^2}$	3.42
$\sqrt{3.3^2 + 1.2^2}$	3.51
$\sqrt{0.1^2 + 0.0^2}$	0.10
$\sqrt{0.1^2 + 0.1^2}$	0.14
$\sqrt{1.4^2 + 1.1^2}$	1.78

Answer

Nearest is (4.7, 1.4) at 0.10 $\Rightarrow$ **versicolor**. With $k = 3$ the vote is versicolor, versicolor, virginica $\Rightarrow$ 2-1, still **versicolor**.

The Same Thing in Code

```
import numpy as np
train = np.array([[1.4,0.2],[1.3,0.2],[4.7,1.4],[4.5,1.5],[6.0,2.5]])
new = np.array([4.6, 1.4])

d = np.sqrt(((train - new)**2).sum(axis=1))
print(np.round(d, 2))
print("nearest row:", d.argmin())
```

```
[3.42 3.51 0.1  0.14 1.78]
nearest row: 2
```

Two things worth noticing

1. KNN does **no training**. It stores the data and does all the work at prediction time.
2. The whole method rests on **distance** — remember that when we get to scaling.

Unsupervised and Reinforcement

- **Clustering** — group similar records: customer segments, disease subtypes. (Unit VII; LA10)

Unsupervised and Reinforcement

- **Clustering** — group similar records: customer segments, disease subtypes. (Unit VII; LA10)
- **Dimensionality reduction** — fewer features, most of the information. (Unit IV)

Unsupervised and Reinforcement

- **Clustering** — group similar records: customer segments, disease subtypes. (Unit VII; LA10)
- **Dimensionality reduction** — fewer features, most of the information. (Unit IV)
- **Anomaly detection** — find what does not fit. (lab experiment 10)

Unsupervised and Reinforcement

- **Clustering** — group similar records: customer segments, disease subtypes. (Unit VII; LA10)
- **Dimensionality reduction** — fewer features, most of the information. (Unit IV)
- **Anomaly detection** — find what does not fit. (lab experiment 10)
- **Reinforcement** — an agent acts, receives reward, and is never told the right answer. Consequences are often delayed by many steps.

Unsupervised and Reinforcement

- **Clustering** — group similar records: customer segments, disease subtypes. (Unit VII; LA10)
- **Dimensionality reduction** — fewer features, most of the information. (Unit IV)
- **Anomaly detection** — find what does not fit. (lab experiment 10)
- **Reinforcement** — an agent acts, receives reward, and is never told the right answer. Consequences are often delayed by many steps.

Unsupervised results are harder to judge

With no ground truth there is no accuracy to compute. If a clustering splits your customers into four groups, nobody can tell you whether those are the *right* four — there is no right. Unit VII spends real time on this.

Naming the Problem

Situation

10 000 past loans marked repaid or defaulted;
score a new applicant
5 years of daily sales; forecast next month
50 000 customer records, no labels; find natural
groups
Card transactions; flag the unusual ones
A warehouse robot learning a route

Family

Task

Naming the Problem

Situation	Family	Task
10 000 past loans marked repaid or defaulted; score a new applicant 5 years of daily sales; forecast next month 50 000 customer records, no labels; find natural groups Card transactions; flag the unusual ones A warehouse robot learning a route	supervised	classification

Naming the Problem

Situation	Family	Task
10 000 past loans marked repaid or defaulted; score a new applicant	supervised	classification
5 years of daily sales; forecast next month	supervised	regression
50 000 customer records, no labels; find natural groups		
Card transactions; flag the unusual ones		
A warehouse robot learning a route		

Naming the Problem

Situation	Family	Task
10 000 past loans marked repaid or defaulted; score a new applicant	supervised	classification
5 years of daily sales; forecast next month	supervised	regression
50 000 customer records, no labels; find natural groups	unsupervised	clustering
Card transactions; flag the unusual ones		
A warehouse robot learning a route		

Naming the Problem

Situation	Family	Task
10 000 past loans marked repaid or defaulted; score a new applicant	supervised	classification
5 years of daily sales; forecast next month	supervised	regression
50 000 customer records, no labels; find natural groups	unsupervised	clustering
Card transactions; flag the unusual ones	unsupervised	anomaly detection
A warehouse robot learning a route		

Naming the Problem

Situation	Family	Task
10 000 past loans marked repaid or defaulted; score a new applicant	supervised	classification
5 years of daily sales; forecast next month	supervised	regression
50 000 customer records, no labels; find natural groups	unsupervised	clustering
Card transactions; flag the unusual ones	unsupervised	anomaly detection
A warehouse robot learning a route	reinforcement	policy learning

Naming the Problem

Situation	Family	Task
10 000 past loans marked repaid or defaulted; score a new applicant	supervised	classification
5 years of daily sales; forecast next month	supervised	regression
50 000 customer records, no labels; find natural groups	unsupervised	clustering
Card transactions; flag the unusual ones	unsupervised	anomaly detection
A warehouse robot learning a route	reinforcement	policy learning

Row 4 repays thought

Fraud *could* be supervised — if you had a reliable label for every past transaction. You do not: confirmed fraud is rare, arrives months late, and is biased by whatever detector was already running. A problem that is supervised in principle is unsupervised in practice, for **operational** reasons, not mathematical ones.

A Model With 100% Training Accuracy

Here it is

Store every training example. When asked to predict, look up the answer.

A Model With 100% Training Accuracy

Here it is

Store every training example. When asked to predict, look up the answer.

Perfect score. Zero learning. It will fail completely on anything it has not seen.

A Model With 100% Training Accuracy

Here it is

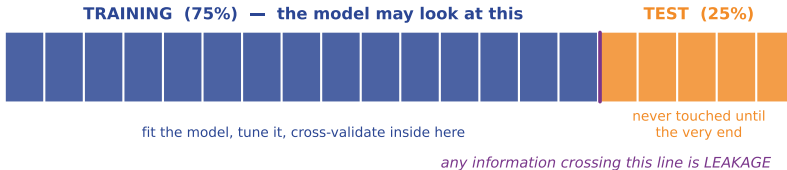
Store every training example. When asked to predict, look up the answer.

Perfect score. Zero learning. It will fail completely on anything it has not seen.

The goal, stated properly

The goal is **never** to fit the data you have. It is to be right about data you have **never seen**. That property is called **generalisation** — and splits, cross-validation and regularization are all machinery for estimating or improving it.

Holding Data Back



The test set is not a tuning signal

Try twenty models, pick the best test score, and that score is no longer honest — you used the test set to make a choice. Split three ways, tune on validation, touch the test set **once**.

Predict First: Which Degree Wins?

We fit polynomials of degree 1, 2, 3 and 12 to 18 noisy points and measure RMSE on held-out data.

Model	Training RMSE	Test RMSE
Degree 1 (straight line)	1.32	
Degree 2 (parabola)	0.89	
Degree 3	0.86	
Degree 12	0.57	

*Before the reveal: which row do you expect to have the lowest **test** error?*

Predict First: Which Degree Wins?

We fit polynomials of degree 1, 2, 3 and 12 to 18 noisy points and measure RMSE on held-out data.

Model	Training RMSE	Test RMSE
Degree 1 (straight line)	1.32	1.60
Degree 2 (parabola)	0.89	1.14
Degree 3	0.86	1.10
Degree 12	0.57	

*Before the reveal: which row do you expect to have the lowest **test** error?*

Predict First: Which Degree Wins?

We fit polynomials of degree 1, 2, 3 and 12 to 18 noisy points and measure RMSE on held-out data.

Model	Training RMSE	Test RMSE
Degree 1 (straight line)	1.32	1.60
Degree 2 (parabola)	0.89	1.14
Degree 3	0.86	1.10
Degree 12	0.57	40.52

*Before the reveal: which row do you expect to have the lowest **test** error?*

Predict First: Which Degree Wins?

We fit polynomials of degree 1, 2, 3 and 12 to 18 noisy points and measure RMSE on held-out data.

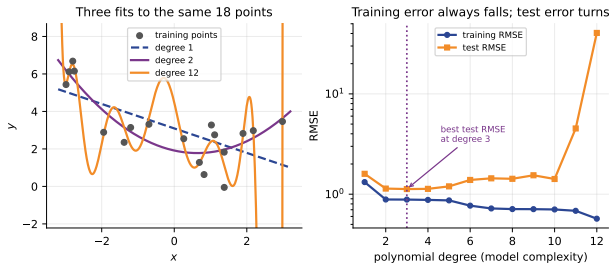
Model	Training RMSE	Test RMSE
Degree 1 (straight line)	1.32	1.60
Degree 2 (parabola)	0.89	1.14
Degree 3	0.86	1.10
Degree 12	0.57	40.52

*Before the reveal: which row do you expect to have the lowest **test** error?*

Read the last row again

Degree 12 is the **best** model by training error and catastrophically the worst by test error — off by 40 where the data spans about 10. Judge on training error and you ship the worst option available.

Overfitting, Seen



The gap is the diagnostic

Both errors high, small gap $\Rightarrow$ **under-fitting**, model too simple.

Tiny training error, large test error $\Rightarrow$ **over-fitting**, model learned the noise.

You want lowest *test* error — which is almost never lowest training error.

Cross-Validation: One Split Is Not Enough

Iris has 150 rows, so a 25% test set is 38 flowers. Two flowers changing sides moves accuracy by five points.

```
from sklearn.model_selection import cross_val_score
scores = cross_val_score(model, X, y, cv=10)
print(scores.round(3))
print(f"mean {scores.mean():.3f} sd {scores.std():.3f}")
```

```
[1.      0.933 1.      0.933 0.867 0.933 0.867 1.      1.      1.      ]
mean 0.953 sd 0.052
```

Look at the spread

Folds range from **0.867 to 1.000**. Reporting “97.4% accuracy” from one lucky split would be true and misleading. The honest summary is 0.953 ± 0.052 — and quoting “95.33%” is false precision.

Predict First: 100 Rows of Pure Noise, Random Labels

```
rng = np.random.default_rng(0)
X = rng.normal(size=(100, 5000)) # pure noise
y = rng.integers(0, 2, 100) # coin flips - NOTHING is predictable

# pick the 20 "best" features using every row, then cross-validate
Xsel = SelectKBest(f_classif, k=20).fit_transform(X, y)
wrong = cross_val_score(KNeighborsClassifier(3), Xsel, y, cv=10).mean()

# selection inside the pipeline, so it happens inside each fold
pipe = make_pipeline(SelectKBest(f_classif, k=20), KNeighborsClassifier(3))
right = cross_val_score(pipe, X, y, cv=10).mean()
```

*The labels are coin flips. What accuracy **must** an honest model get?*

Predict First: 100 Rows of Pure Noise, Random Labels

```
rng = np.random.default_rng(0)
X = rng.normal(size=(100, 5000)) # pure noise
y = rng.integers(0, 2, 100) # coin flips - NOTHING is predictable

# pick the 20 "best" features using every row, then cross-validate
Xsel = SelectKBest(f_classif, k=20).fit_transform(X, y)
wrong = cross_val_score(KNeighborsClassifier(3), Xsel, y, cv=10).mean()

# selection inside the pipeline, so it happens inside each fold
pipe = make_pipeline(SelectKBest(f_classif, k=20), KNeighborsClassifier(3))
right = cross_val_score(pipe, X, y, cv=10).mean()
```

*The labels are coin flips. What accuracy **must** an honest model get?*

```
selection before the split : 0.81
selection inside the fold  : 0.50
```

Data Leakage

81% accuracy on random noise

The selector examined all 5000 columns **including the test rows' labels** and kept the twenty that correlated by chance. Those twenty then looked predictive during cross-validation — because the selection had already peeked.

Definition

Data leakage is the use, at training time, of information that would not be available at prediction time.

The rule that makes it automatic

Any step that **learns something from the data** — scaling, imputation, feature selection, encoding — must be fitted on the training portion only. Put every such step inside a Pipeline and let scikit-learn handle the fold boundaries.

Three Libraries, Three Jobs

scikit-learn — models, preprocessing, evaluation, pipelines

pandas — tabular data: load, clean, reshape, group

NumPy — the ndarray and fast array arithmetic

Everything above rests on the layer below

A pandas DataFrame holds NumPy arrays; a scikit-learn estimator consumes NumPy arrays. Understanding the bottom layer is what lets you debug the top one.

By Hand: MAE, MSE and RMSE

True values $y = (3, 1, 4)$, predictions $\hat{y} = (2.5, 1.5, 4.5)$.

Errors: $y - \hat{y} = (0.5, -0.5, -0.5)$.

By Hand: MAE, MSE and RMSE

True values $y = (3, 1, 4)$, predictions $\hat{y} = (2.5, 1.5, 4.5)$.

Errors: $y - \hat{y} = (0.5, -0.5, -0.5)$.

$$\text{MAE} = \frac{0.5 + 0.5 + 0.5}{3} = 0.5$$

$$\text{MSE} = \frac{0.25 + 0.25 + 0.25}{3} = 0.25$$

$$\text{RMSE} = \sqrt{0.25} = 0.5$$

By Hand: MAE, MSE and RMSE

True values $y = (3, 1, 4)$, predictions $\hat{y} = (2.5, 1.5, 4.5)$.

Errors: $y - \hat{y} = (0.5, -0.5, -0.5)$.

$$\text{MAE} = \frac{0.5 + 0.5 + 0.5}{3} = 0.5$$

$$\text{MSE} = \frac{0.25 + 0.25 + 0.25}{3} = 0.25$$

$$\text{RMSE} = \sqrt{0.25} = 0.5$$

Now change one

Set $\hat{y}_1 = 0$, so errors are
(3, -0.5, -0.5):

$$\text{MAE} = 1.333$$

$$\text{RMSE} = 1.780$$

By Hand: MAE, MSE and RMSE

True values $y = (3, 1, 4)$, predictions $\hat{y} = (2.5, 1.5, 4.5)$.

Errors: $y - \hat{y} = (0.5, -0.5, -0.5)$.

$$\text{MAE} = \frac{0.5 + 0.5 + 0.5}{3} = 0.5$$

$$\text{MSE} = \frac{0.25 + 0.25 + 0.25}{3} = 0.25$$

$$\text{RMSE} = \sqrt{0.25} = 0.5$$

Now change one

Set $\hat{y}_1 = 0$, so errors are
 $(3, -0.5, -0.5)$:

$$\text{MAE} = 1.333$$

$$\text{RMSE} = 1.780$$

Why they diverge

Squaring punishes the one big error far more than the two small ones. That asymmetry is exactly why the choice of loss function matters — and it is the whole of Unit II.

The Same Thing in NumPy

```
y = np.array([3.0, 1.0, 4.0])
yhat = np.array([2.5, 1.5, 4.5])

print("MAE ", np.mean(np.abs(y - yhat)))
print("MSE ", np.mean((y - yhat)**2))
print("RMSE", np.sqrt(np.mean((y - yhat)**2)))
```

```
MAE 0.5
MSE 0.25
RMSE 0.5
```

You just wrote the first loss function of the course

`np.mean((y - yhat)**2)` is the **Mean Squared Error**. Every optimiser in Unit III exists to make that number smaller. In a sense the whole course is an elaboration of this one line.

Predict First: How Much Faster Is Vectorisation?

Python loop

```
total = 0.0
for i in range(len(y)):
    d = y[i] - yhat[i]
    total += d * d
mse = total / len(y)
```

NumPy

```
mse = np.mean((y - yhat)**2)
```

One million elements. Guess the ratio.

Predict First: How Much Faster Is Vectorisation?

Python loop

```
total = 0.0
for i in range(len(y)):
    d = y[i] - yhat[i]
    total += d * d
mse = total / len(y)
```

```
loop :    162.9 ms
numpy :     1.85 ms
ratio :      88x
```

NumPy

```
mse = np.mean((y - yhat)**2)
```

One million elements. Guess the ratio.

Predict First: How Much Faster Is Vectorisation?

Python loop

```
total = 0.0
for i in range(len(y)):
    d = y[i] - yhat[i]
    total += d * d
mse = total / len(y)
```

```
loop :    162.9 ms
numpy :     1.85 ms
ratio :      88x
```

The loop runs in the interpreter, one element at a time. NumPy dispatches to compiled C walking a contiguous block. The gap **widens** with array size.

NumPy

```
mse = np.mean((y - yhat)**2)
```

One million elements. Guess the ratio.

Predict First: What Does This Print?

```
X = np.array([[1., 2.],  
              [3., 4.],  
              [5., 6.]])  
col_means = X.mean(axis=0)  
print(col_means)  
print(X - col_means)
```

axis=0 collapses the rows. Write down both answers before advancing.

Predict First: What Does This Print?

```
X = np.array([[1., 2.],  
              [3., 4.],  
              [5., 6.]])  
col_means = X.mean(axis=0)  
print(col_means)  
print(X - col_means)
```

axis=0 collapses the rows. Write down both answers before advancing.

```
[3. 4.]  
[[-2. -2.]  
 [ 0.  0.]  
 [ 2.  2.]]
```

Predict First: What Does This Print?

```
X = np.array([[1., 2.],  
              [3., 4.],  
              [5., 6.]])  
col_means = X.mean(axis=0)  
print(col_means)  
print(X - col_means)
```

axis=0 collapses the rows. Write down both answers before advancing.

```
[3. 4.]  
[[-2. -2.]  
 [ 0.  0.]  
 [ 2.  2.]]
```

Two ideas at once

Broadcasting: a (3,2) array minus a (2,) array works — the small one is stretched down the rows. And you have just **centred a dataset** — the first half of what StandardScaler does.

pandas: the Four Calls, Every Time

```
df.head() # look at it first, always  
df.info() # dtypes and non-null counts: finds text stored as numbers  
df.describe() # numeric summary  
df.isna().sum() # where is the data missing?
```

	sepal length	sepal width	petal length	petal width
count	150.00	150.00	150.00	150.00
mean	5.84	3.06	3.76	1.20
std	0.83	0.44	1.77	0.76
min	4.30	2.00	1.00	0.10
max	7.90	4.40	6.90	2.50

Read the table, do not skip it

Petal length has sd **1.77**; petal width has **0.76** — more than double. Sepal width spans 2.4 cm; petal length spans 5.9 cm. Hold that thought for two slides.

scikit-learn: One API for Every Model

```
est.fit(X, y) # learn  
est.predict(X) # apply  
est.score(X, y) # evaluate
```

```
tr.fit(X) # transformers  
tr.transform(X)
```

Why this matters

Hundreds of models, one interface. Swapping KNN for an SVM changes **one line**. Learn it once here and every model in Units V–VII is already familiar.

The rule you must not break

Call `fit` on training data only — never on the test set, not even for scaling. We saw what that does: 81% on pure noise.

An Entire Workflow in Twelve Lines

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.pipeline import make_pipeline
```

```
X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=0, stratify=y)
```

```
model = make_pipeline(StandardScaler(), KNeighborsClassifier(n_neighbors=5))
model.fit(X_train, y_train)
print("test accuracy:", model.score(X_test, y_test))
```

```
test accuracy: 0.9736842105263158
```

random_state makes it reproducible • stratify=y keeps class balance • make_pipeline fits the scaler *inside* training only.

One Dataset, One Seed

The convention for this whole course

- **One dataset per lecture** where possible — here `load_iris`, bundled with scikit-learn. Nothing downloads.
- **One seed: 0.** Every split `random_state=0`, every generator `np.random.default_rng(0)`.
- The seed is **shown in the listing**, never hidden in a helper.

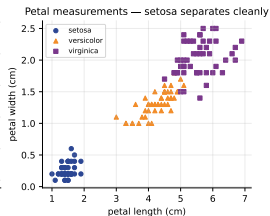
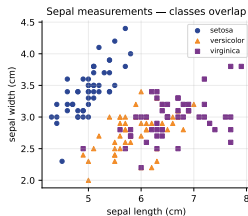
Why you should care

Every number in the notes then reproduces *exactly* on your machine. If yours differs, the cause is your code or your library versions — not luck. An unseeded result is a number nobody can check, including you, tomorrow.

A seed fixes chance. It does *not* fix wall-clock timings — those depend on your processor, so never quote one as a fact.

Accuracy Hides Things. Look at the Matrix.

	pred setosa	pred versicolor	pred virginica
true setosa	13	0	0
true versicolor	0	13	0
true virginica	0	1	11



One error in 38

A virginica called versicolor — exactly the pair the figure shows touching. The model failed precisely where the data is genuinely ambiguous.

A confusion matrix tells you *which* mistakes are made. An accuracy figure never does.

Predict First: What Happens Without Scaling?

One feature decides the class, on a range of **0–1**. A second feature is pure noise, on a range of **0–10 000**. We fit KNN to both.

Model	Test accuracy
KNeighborsClassifier(5) on raw features make_pipeline(StandardScaler(), KNN(5))	

Guess both numbers before advancing.

Predict First: What Happens Without Scaling?

One feature decides the class, on a range of **0–1**. A second feature is pure noise, on a range of **0–10 000**. We fit KNN to both.

Model	Test accuracy
KNeighborsClassifier(5) on raw features	0.50
make_pipeline(StandardScaler(), KNN(5))	0.96

Guess both numbers before advancing.

Predict First: What Happens Without Scaling?

One feature decides the class, on a range of **0–1**. A second feature is pure noise, on a range of **0–10 000**. We fit KNN to both.

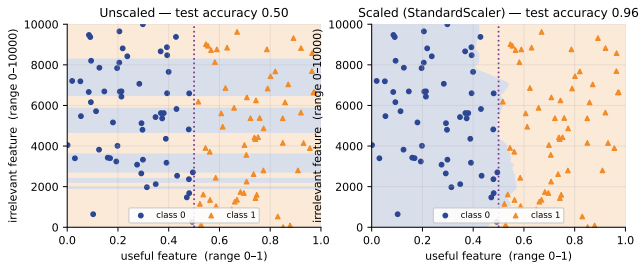
Model	Test accuracy
KNeighborsClassifier(5) on raw features	0.50
make_pipeline(StandardScaler(), KNN(5))	0.96

Guess both numbers before advancing.

0.50 is a coin flip

Euclidean distance squares each difference and adds. A gap of 3000 in the noise feature contributes 9,000,000; a gap of 0.5 in the useful feature contributes 0.25. The useful feature is not outweighed — it is **invisible**.

Scaling, Seen



Which methods care?

Need scaling: anything measuring distance (KNN, SVM with RBF, K-means) or using gradient descent (Units II–III).

Do not: tree-based methods — they split one feature at a time, so the units never interact.

Choosing a Hyperparameter by Measurement

```
k=1    cv accuracy 0.953
k=3    cv accuracy 0.953
k=5    cv accuracy 0.953
k=7    cv accuracy 0.953
k=9    cv accuracy 0.953
k=15   cv accuracy 0.960
k=25   cv accuracy 0.947
k=50   cv accuracy 0.880
```

Choosing a Hyperparameter by Measurement

k=1	cv accuracy 0.953
k=3	cv accuracy 0.953
k=5	cv accuracy 0.953
k=7	cv accuracy 0.953
k=9	cv accuracy 0.953
k=15	cv accuracy 0.960
k=25	cv accuracy 0.947
k=50	cv accuracy 0.880

Two lessons in one table

1. Flat from $k=1$ to $k=9$. On an easy dataset, tuning barely matters — a student reporting “I tuned k and got a big improvement” has probably fooled themselves.
2. At $k=50$ it collapses. With 50 flowers per species, a 50-neighbour vote reaches across class boundaries.

Choosing a Hyperparameter by Measurement

k=1	cv accuracy	0.953
k=3	cv accuracy	0.953
k=5	cv accuracy	0.953
k=7	cv accuracy	0.953
k=9	cv accuracy	0.953
k=15	cv accuracy	0.960
k=25	cv accuracy	0.947
k=50	cv accuracy	0.880

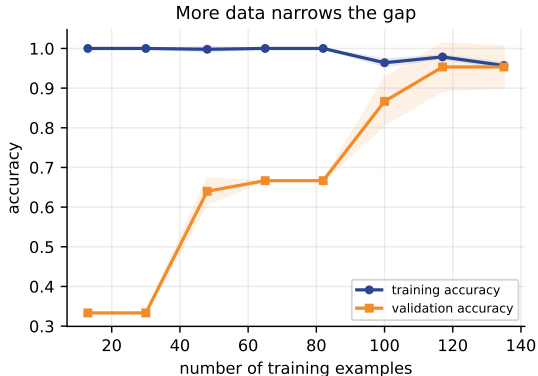
Two lessons in one table

1. Flat from $k=1$ to $k=9$. On an easy dataset, tuning barely matters — a student reporting “I tuned k and got a big improvement” has probably fooled themselves.
2. At $k=50$ it collapses. With 50 flowers per species, a 50-neighbour vote reaches across class boundaries.

k too small $\Rightarrow$ fits noise.

k too large $\Rightarrow$ smooths across classes.

More Data, or a Better Model?



Read the right-hand edge

The curves have converged by 120 examples. More Iris flowers would not help — the limit is the model or the features, not the data.

When they are still far apart

More data is the cheapest improvement available. Plot this before reaching for a fancier model.

Before the First Lab

Set up your environment

1. Install Python 3.11+ (Anaconda or venv).
2. `pip install numpy pandas scikit-learn matplotlib seaborn jupyter`
3. Run the twelve-line Iris script.
4. Print the version of each library.

Lab assignment LA1

Experiments 1–2: environment setup, then preprocessing and EDA. Submission date announced in the lab; quiz LQ1 follows.

Lab quizzes test *your* code

60 minutes at the machine, reading, tracing, debugging and modifying what *you* submitted. Copied code is hard to defend.

Summary

- ML suits problems where you have **examples of the answer** but cannot **state the rule**.

Sources: Müller and Guido, *Introduction to Machine Learning with Python*, Ch. 1.; Bishop, *Pattern Recognition and Machine Learning*, Ch. 1.; Murphy, *Probabilistic Machine Learning: An Introduction*, MIT Press, 2022, Ch. 1.; Mitchell, *Machine Learning*, McGraw Hill, 1997, Ch. 1.; Scikit-learn documentation, accessed 2026-08-04.

Summary

- ML suits problems where you have **examples of the answer** but cannot **state the rule**.
- Mitchell's T , P , E forces you to name the performance measure first. Most failures are P failures.

Sources: Müller and Guido, *Introduction to Machine Learning with Python*, Ch. 1.; Bishop, *Pattern Recognition and Machine Learning*, Ch. 1.; Murphy, *Probabilistic Machine Learning: An Introduction*, MIT Press, 2022, Ch. 1.; Mitchell, *Machine Learning*, McGraw Hill, 1997, Ch. 1.; Scikit-learn documentation, accessed 2026-08-04.

Summary

- ML suits problems where you have **examples of the answer** but cannot **state the rule**.
- Mitchell's T , P , E forces you to name the performance measure first. Most failures are P failures.
- **Supervised** (regression, classification), **unsupervised** (clustering, dim. reduction, anomaly), **reinforcement** (policy from reward).

Sources: Müller and Guido, *Introduction to Machine Learning with Python*, Ch. 1.; Bishop, *Pattern Recognition and Machine Learning*, Ch. 1.; Murphy, *Probabilistic Machine Learning: An Introduction*, MIT Press, 2022, Ch. 1.; Mitchell, *Machine Learning*, McGraw Hill, 1997, Ch. 1.; Scikit-learn documentation, accessed 2026-08-04.

Summary

- ML suits problems where you have **examples of the answer** but cannot **state the rule**.
- Mitchell's T , P , E forces you to name the performance measure first. Most failures are P failures.
- **Supervised** (regression, classification), **unsupervised** (clustering, dim. reduction, anomaly), **reinforcement** (policy from reward).
- The goal is never to fit the training data — it is to **generalise**. The train–test **gap** is your diagnostic.

Sources: Müller and Guido, *Introduction to Machine Learning with Python*, Ch. 1.; Bishop, *Pattern Recognition and Machine Learning*, Ch. 1.; Murphy, *Probabilistic Machine Learning: An Introduction*, MIT Press, 2022, Ch. 1.; Mitchell, *Machine Learning*, McGraw Hill, 1997, Ch. 1.; Scikit-learn documentation, accessed 2026-08-04.

Summary

- ML suits problems where you have **examples of the answer** but cannot **state the rule**.
- Mitchell's T , P , E forces you to name the performance measure first. Most failures are P failures.
- **Supervised** (regression, classification), **unsupervised** (clustering, dim. reduction, anomaly), **reinforcement** (policy from reward).
- The goal is never to fit the training data — it is to **generalise**. The train–test **gap** is your diagnostic.
- One split on a small dataset is unreliable. Cross-validate and **quote the spread**.

Sources: Müller and Guido, *Introduction to Machine Learning with Python*, Ch. 1.; Bishop, *Pattern Recognition and Machine Learning*, Ch. 1.; Murphy, *Probabilistic Machine Learning: An Introduction*, MIT Press, 2022, Ch. 1.; Mitchell, *Machine Learning*, McGraw Hill, 1997, Ch. 1.; Scikit-learn documentation, accessed 2026-08-04.

Summary

- ML suits problems where you have **examples of the answer** but cannot **state the rule**.
- Mitchell's T , P , E forces you to name the performance measure first. Most failures are P failures.
- **Supervised** (regression, classification), **unsupervised** (clustering, dim. reduction, anomaly), **reinforcement** (policy from reward).
- The goal is never to fit the training data — it is to **generalise**. The train–test **gap** is your diagnostic.
- One split on a small dataset is unreliable. Cross-validate and **quote the spread**.
- **Leakage** gave 81% on pure noise. Put every fitted step inside a Pipeline.

Sources: Müller and Guido, *Introduction to Machine Learning with Python*, Ch. 1.; Bishop, *Pattern Recognition and Machine Learning*, Ch. 1.; Murphy, *Probabilistic Machine Learning: An Introduction*, MIT Press, 2022, Ch. 1.; Mitchell, *Machine Learning*, McGraw Hill, 1997, Ch. 1.; Scikit-learn documentation, accessed 2026-08-04.

Summary

- ML suits problems where you have **examples of the answer** but cannot **state the rule**.
- Mitchell's T , P , E forces you to name the performance measure first. Most failures are P failures.
- **Supervised** (regression, classification), **unsupervised** (clustering, dim. reduction, anomaly), **reinforcement** (policy from reward).
- The goal is never to fit the training data — it is to **generalise**. The train–test **gap** is your diagnostic.
- One split on a small dataset is unreliable. Cross-validate and **quote the spread**.
- **Leakage** gave 81% on pure noise. Put every fitted step inside a Pipeline.
- Distance- and gradient-based methods need **scaling**. Trees do not.

Sources: Müller and Guido, *Introduction to Machine Learning with Python*, Ch. 1.; Bishop, *Pattern Recognition and Machine Learning*, Ch. 1.; Murphy, *Probabilistic Machine Learning: An Introduction*, MIT Press, 2022, Ch. 1.; Mitchell, *Machine Learning*, McGraw Hill, 1997, Ch. 1.; Scikit-learn documentation, accessed 2026-08-04.

Next Lecture, and Where to Read More

Next — L2: Loss functions (Unit II)

Putting a number on how wrong a prediction is: MSE, MAE and Huber for regression; cross-entropy for classification; hinge and triplet for margins. That number is what everything else minimises.

Read

Müller & Guido Ch. 1 • Bishop Ch. 1 §1.1

ISLP Ch. 1–2 • Murphy PML Ch. 1

Watch (free)

StatQuest, *A Gentle Introduction to ML*

youtu.be/Gv9_4yMHFhI

Courses (free)

NPTEL, *Introduction to Machine Learning*

nptel.ac.in/courses/106106139

Kaggle Learn, *Intro to ML*

Interactive

mlu-explain.github.io

The notes carry the full, linked reference list — see [notes.pdf](#), final section.



Thank You

